

EE 585: Probabilistic Robotics - Homework 1 Report

Author:

Çağdaş Güven

Date: November 4, 2025

Introduction

This report details the work completed for EE 585 Homework 1. The primary objective is twofold: first, to review and select a modern robotics simulator suitable for probabilistic robotics tasks, and second, to implement a multi-robot 'leader-follower' scenario to demonstrate the simulator's capabilities.

The chosen simulation platform for this project is **Gazebo Sim (v6.16)**, integrated within a **ROS 2 Humble** ecosystem. To ensure a consistent and reproducible development environment, the entire project is managed within a **Docker** container orchestrated by **VS Code Dev Containers**. This setup allows for a sandboxed environment with all necessary dependencies (ROS 2, Gazebo, Python libraries) pre-installed and configured.

Q1: Simulator Review - Gazebo Sim

For this assignment, Gazebo Sim (formerly Ignition Gazebo) version 6.16 was selected. This review assesses its capabilities against the criteria required for probabilistic robotics, specifically its deep integration with ROS 2 Humble for sensor data-passing and robot control.

a. Ability to simulate the kinematics and/or dynamics of a differential drive (wheeled) robot

Gazebo Sim utilizes Simulation Description Format (SDF) to define both robots and their environments. Unlike URDF, which primarily focuses on robot kinematics, SDF offers a comprehensive world description. This allows for the definition of non-robot entities such as lights and terrain, and crucially, enables the specification of physical properties like friction and joint dynamics, features absent in standard URDF.

To facilitate physics simulation, the `ignition-gazebo-physics-system` plugin is integrated into the world's SDF. This provides granular control over the dynamic properties of each robot link, which are directly defined within the robot's model SDF. For the `diff_drive` robot, these properties include mass, inertia, visual geometry, and collision geometry.

We can define our robot and its environment using a hierarchical ordered structure of tags. The top-level tag is `link`, which contains information about individual parts of the robot. Within each `link`, we find tags like `pose`, `inertial`, `visual`, and `collision`.

The `pose` tag defines the position and orientation of the link, for example, `-0.151427 -0.0175 0 -0 0` for the chassis. The `inertial` tag describes the physical properties, including mass (e.g., 1.14395 kg for the chassis) and inertia tensors with components like `ixx` (e.g., 0.126164), `ixy`, `ixz`, `iyy` (e.g., 0.416519), `iyz`, and `izz` (e.g., 0.481014).

The `visual` tag (e.g., `visual_chassis`) is used for rendering, while the `collision` tag (e.g., `collision_chassis`) defines the physical interaction properties. Inside `collision`, we have `geometry` which can contain shapes like `sphere` (e.g., with a radius of 0.3 for the wheel). The `surface` tag within `collision` is crucial for defining interaction characteristics. It contains `friction` and `contact` tags.

The `friction` tag, specifically for `ode` (Open Dynamics Engine), includes parameters like `mu` (e.g., 1.0) and `mu2` (e.g., 0.8), which represent the directional friction values. The `slip1` (e.g., 0.05) and `slip2` (e.g., 0.05) tags define the "real" noise caused by slippage.

Finally, the `contact` tag, also for `ode`, includes `min_depth` (e.g., 0.001) and `kp` (e.g., 1e8) for contact resolution. Many of these tags are self-explanatory, providing a clear way to model robot components and their interactions within a simulated environment.

The capacity to define these elaborate physical properties, such as `<friction>` and `<mass>`, is essential for the simulator to accurately model the kinematics and dynamics of a differential drive robot.

b. Ability to command the linear and angular velocities of the robot

The simulation provides a direct interface for robot control via ROS 2 topics. This is achieved using the `gz-sim-diff-drive-system` plugin, which is included in the robot's SDF file. This plugin subscribes to a ROS 2 topic of type `geometry_msgs/msg/Twist`.

The `geometry_msgs/msg/Twist` interface is the standard ROS 2 message for velocity commands:

```
root@pc:/workspaces/585_Probabilistic_Robotics# ros2 interface show
geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

Publishing to the topic specified by the plugin (e.g., /follower/cmd_vel) allows for independent control of the robot's linear velocity (m/s) via linear.x and angular velocity (rad/s) via angular.z. This was verified using the standard ros2_teleop package, as shown in Figure 1.

```
root@pc:/workspaces/585_Probabilistic_Robotics# ros2 run teleop_twist_keyboard teleop_twist_keyboard -
-rs-args -r cmd_vel:=/model/vehicle_blue/cmd_vel

This node takes keypresses from the keyboard and publishes them
as Twist/TwistStamped messages. It works best with a US keyboard layout.
-----
Moving around:
  u   i   o
  j   k   l
  m   ,   .

For Holonomic mode (strafing), hold down the shift key:
-----
  U   I   O
  J   K   L
  M   <   >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.5      turn 1.0
```

Figure 1: Using the teleop_twist_key node to publish geometry_msgs/msg/Twist commands to the /leader/cmd_vel topic, demonstrating direct velocity control.

This capability extends to multi-robot scenarios. As shown in Figure 2, we instantiated two robots, 'leader' and 'follower'. Each is spawned within a unique ROS 2 namespace, allowing them to be commanded independently by publishing to their respective `/leader/cmd_vel` and `/follower/cmd_vel` topics.

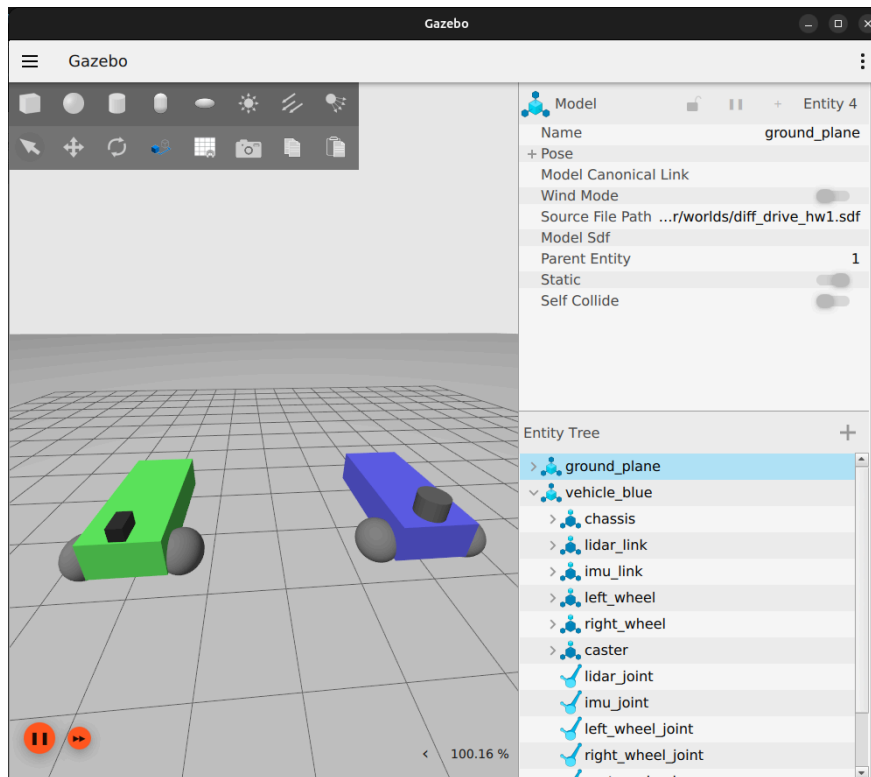


Figure 2: Screenshot from Gazebo showing both the 'leader' and 'follower' robots. The GUI's "Entity Tree" and "Component Inspector" panels (left) confirm both models are spawned and their components are accessible.

c. Ability to simulate “real” noise into the motion kinematics/dynamics of the robot

The simulator can model physical motion noise separately from sensor noise. This fulfills the requirement to simulate "non-modeled dynamics" like wheel-ground interaction and slippage.

Instead of adding noise to a sensor, this is modeled directly in the physics simulation. Within the robot's SDF, the wheel's `<collision>` tag contains a `<surface>` block. By defining the ode (Open Dynamics Engine) friction and slip parameters, we can simulate a non-ideal contact, causing the robot's actual path to drift from a perfect command.

Wheel Link (with friction and slip):

```
<collision name='collision_right_wheel'>
  <geometry>
    <sphere>
      <radius>0.3</radius>
    </sphere>
  </geometry>
  <surface>
    <friction>
      <ode>
        <mu>1.0</mu> <mu2>0.8</mu2> <slip1>0.05</slip1> <slip2>0.05</slip2> </ode>
      </friction>
    </contact>
    <contact>
      <ode>
        <min_depth>0.001</min_depth>
        <kp>1e8</kp>
      </ode>
    </contact>
  </surface>
</collision>
```

This physical drift is demonstrated in Figure 3, which plots the robot's ground-truth path while attempting to drive in a perfect circle. The path with `slip` parameters enabled (noisy) clearly deviates from the ideal path, showing the cumulative effect of this "real" motion noise.

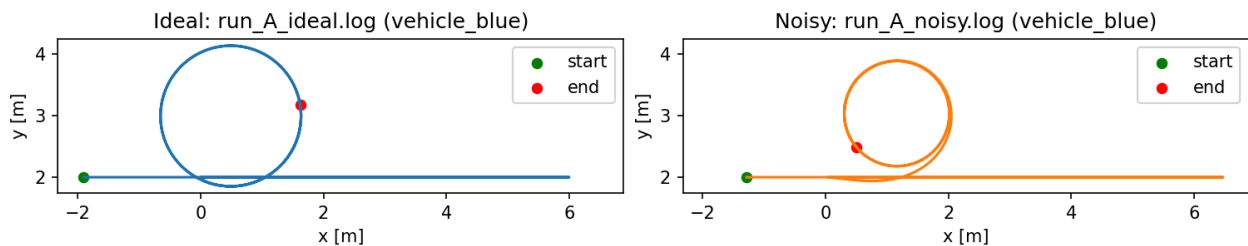


Figure 3: A plot comparing the Ground Truth pose of the robot in an ideal simulation (blue, perfect circle) vs. a simulation with wheel slip enabled (red, drifting path).

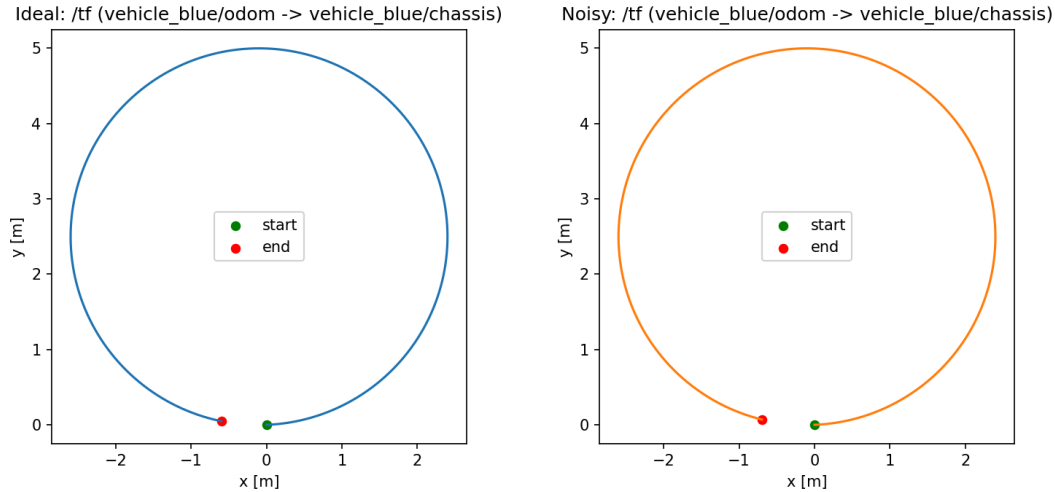


Figure 4: A plot showing ideal and noisy run from robots perspective, which leads to kidnapped robot scenario because robot thinks as it moves in perfect order

d. Provide distance sensor data, especially a “distance scanner” sensor

The simulator provides multiple sensors for distance measurement. For this project, we successfully added and configured two key distance sensors to our robot model:

1. **LIDAR (Laser Scanner):** A 2D laser scanner (gz-sim-sensors-system with type `gpu_lidar`) was added. It publishes `sensor_msgs/msg/LaserScan` messages (e.g., on `/lidar/scan`), providing an array of range measurements as a function of bearing.
2. **Depth Camera:** An RGBD camera (gz-sim-sensors-system with type `rgbd_camera`) was also added. This sensor publishes distance information as both a `sensor_msgs/msg/Image` (a 32-bit float depth map) and a `sensor_msgs/msg/PointCloud2`.

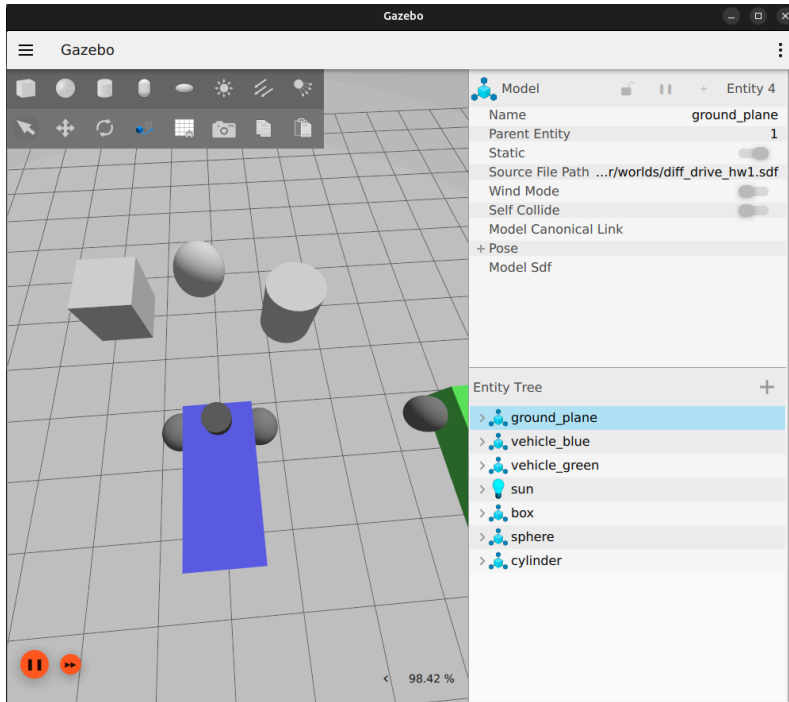


Figure 5 shows the LIDAR sensor data being published from Gazebo (left) and visualized in RViz2 (right). The LaserScan message (visualized as red dots) correctly identifies the surrounding objects.

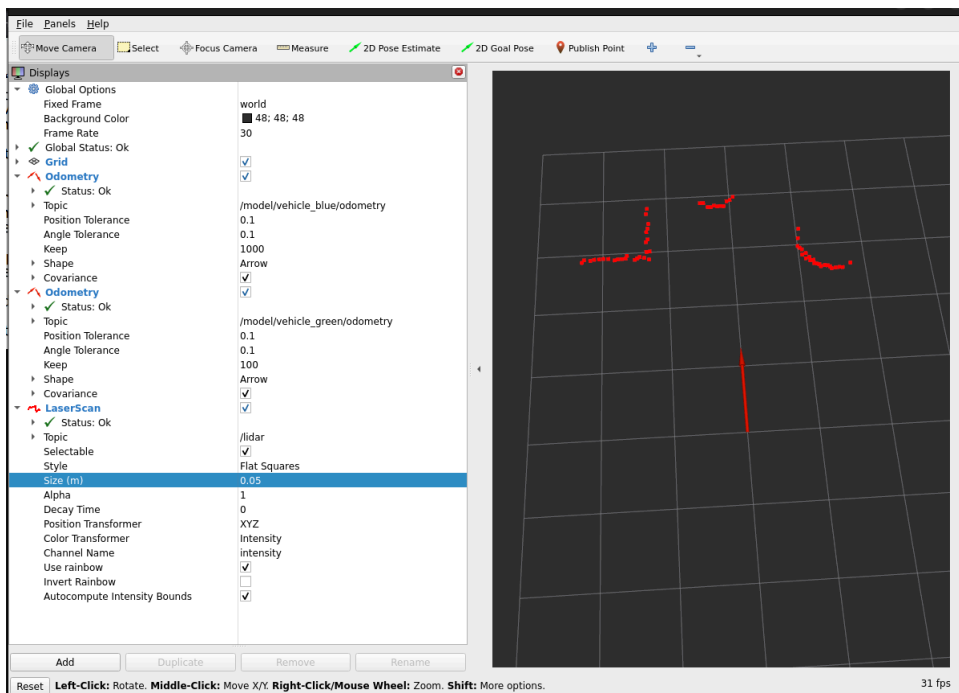


Figure 6: RViz2 visualizing the sensor_msgs/msg/LaserScan data from the 'follower' robot's LIDAR (topic: /lidar/scan). The scan data clearly perceives the objects placed in front.

e. Ability to define a map environment with pre-defined "landmarks" and/or geometric "obstacles"

Gazebo Sim supports a highly flexible definition of map environments, which is crucial for both navigation (obstacles) and localization (landmarks). We demonstrated this capability in three distinct ways:

1. **Static World Definition:** The primary testing environment was defined in the `diff_drive_hw1.sdf` world file. This includes static obstacles such as the four walls enclosing the area.
2. **Dynamic World Features:** The world SDF also defines dynamic obstacles, such as a "swing_door" model with a hinge joint. This door can be manipulated at runtime by calling the `/world/demo/set_pose` service (provided by `ros_gz_bridge` and `ros_gz_sim`) to update the pose of its "leaf" link, simulating a real-world dynamic environment.
3. **Dynamic Landmark Spawning:** For adding landmarks at arbitrary locations during runtime, two methods were confirmed:
 - **GUI-based:** By including the `ignition::gazebo::systems::UserCommands` plugin in the world SDF, simple geometric primitives (Box, Sphere, Cylinder, etc.) can be dragged and dropped from the GUI menu, as shown in Figure 3.
 - **Programmatic (ROS 2):** For reproducible tests, the `ros_gz_sim` node `spawn_entity` was used. This script calls the ROS 2 service `/world/<world_name>/create` to spawn models from SDF files (e.g., a pre-defined "landmark" model) at a specific pose.

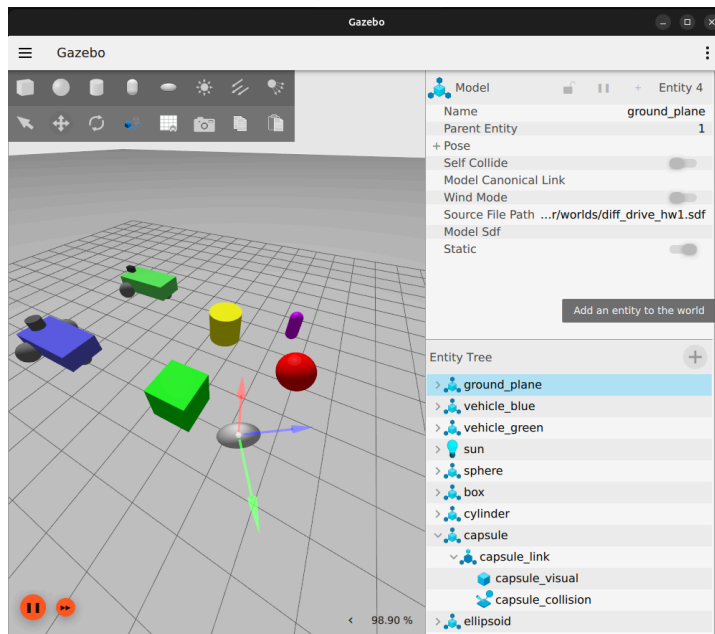


Figure 7: Simple geometric primitives (box, sphere, cylinder, capsule, ellipsoid) dynamically added to the simulation at runtime using the Gazebo GUI.

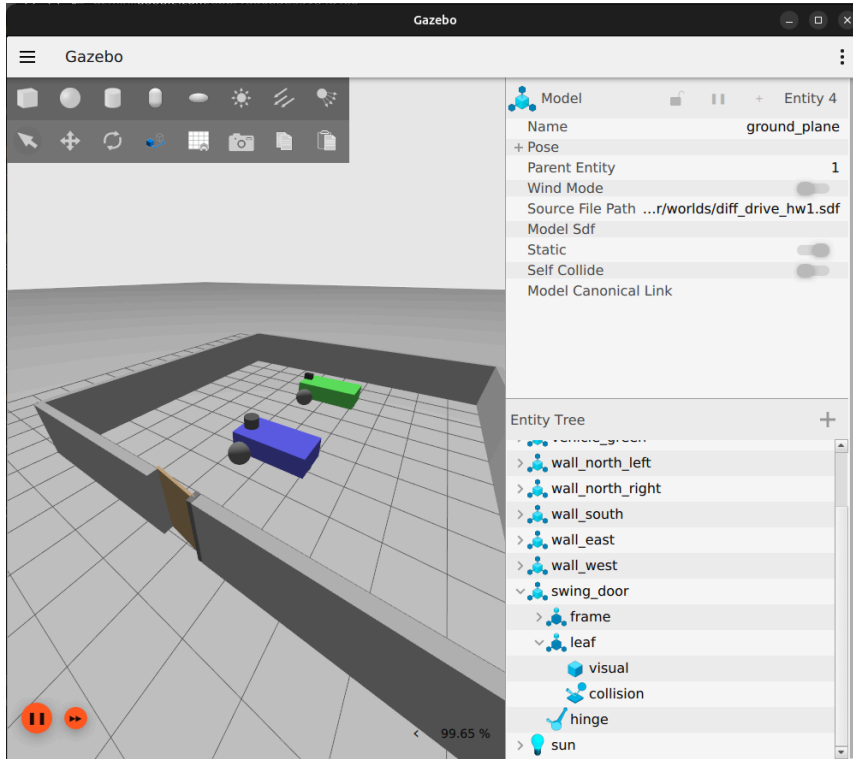


Figure 8: Custom map environment defined in the world SDF, featuring static walls and a dynamic, jointed door (center-left).

f. Provide wheel odometry data from the robot

The `gz-sim-diff-drive-system` plugin, in addition to subscribing to velocity commands, also simulates and publishes the robot's odometry. It publishes `nav_msgs/msg/Odometry` messages (e.g., `/follower/odom`). This message contains the robot's pose estimate based on simulated wheel encoders (which includes the noise specified in section c) and its estimated twist. This data serves as the primary input for motion models in probabilistic filters. Access to ground truth is also available via the `/world/<world_name>/pose/info` topic, which is essential for evaluating the performance of any estimation algorithm by comparing its output to the true state.

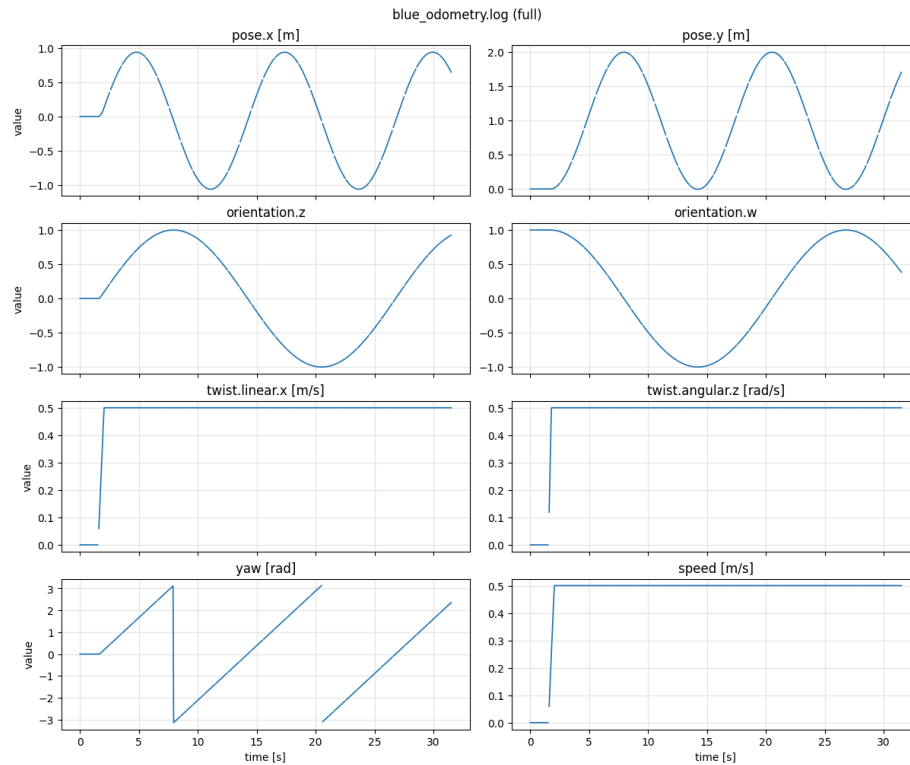


Figure 9: A plot showing the output of 30 seconds of data collected from robot doing circular static movement

g. Provide camera image data from the robot

Yes, the `gz-sim-sensors-system` plugin (with a sensor of type `rgbd_camera` or `camera`) provides standard ROS 2 image data. For our project, the RGBD camera sensor publishes:

- `sensor_msgs/msg/Image` on the `/camera/image_raw` topic for the RGB video stream.
- `sensor_msgs/msg/Image` on the `/depth_camera/image_raw` topic for the depth map.
- `sensor_msgs/msg/PointCloud2` on `/depth_camera/points` for the 3D point cloud.
- `sensor_msgs/msg/CameraInfo` on corresponding topics, providing the camera's intrinsic calibration parameters.

This data is used in our Q2 task for visual tracking. Figure 8 provides a side-by-side comparison of the Gazebo simulator view and the corresponding data received and visualized in RViz2.

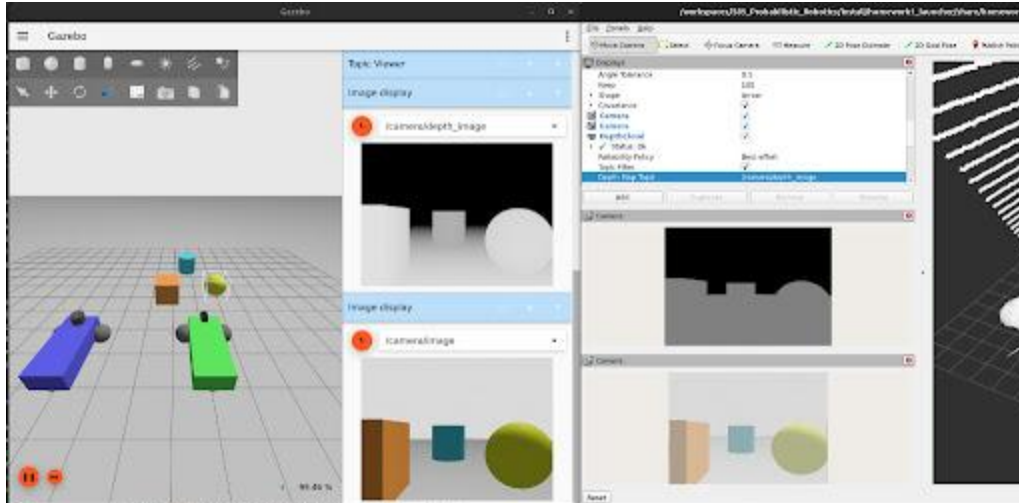


Figure 10: (Left) Gazebo simulation view. (Right) RViz2 visualizing the data from the RGBD sensor. This includes the `sensor_msgs/msg/Image` (bottom-left panel) and the `sensor_msgs/msg/PointCloud2` (main panel), which is required for 3D depth visualization.

h. Provide Inertial sensor (IMU) data from the robot

An IMU was successfully added to the robot model using the `gz-sim-sensors-system` plugin with a sensor of type `imu`. This plugin publishes standard `sensor_msgs/msg/Imu` messages (e.g., on `/imu`) containing the robot's orientation, angular velocity, and linear acceleration.

A sample measurement from the `/imu` topic (while the robot was moving) shows the raw data structure:

```
entity_name: "vehicle_blue::imu_link::imu_sensor"
orientation {
  x: 9.8246646822547683e-17
  y: -2.9332158445138906e-10
  z: -2.2694752594543663e-17
  w: 1
}
angular_velocity {
  x: 0.0038255768699483475
  y: -0.005582150090951617
  z: -0.00096998097028895721
}
linear_acceleration {
  x: 0.012624294593205148
  y: 0.023535098037162184
  z: 9.7987541316300586
}
```

This data was logged for 30 seconds while the robot was in motion and plotted to visualize the sensor's output over time (Figure 11). This data is available for use in sensor fusion algorithms.

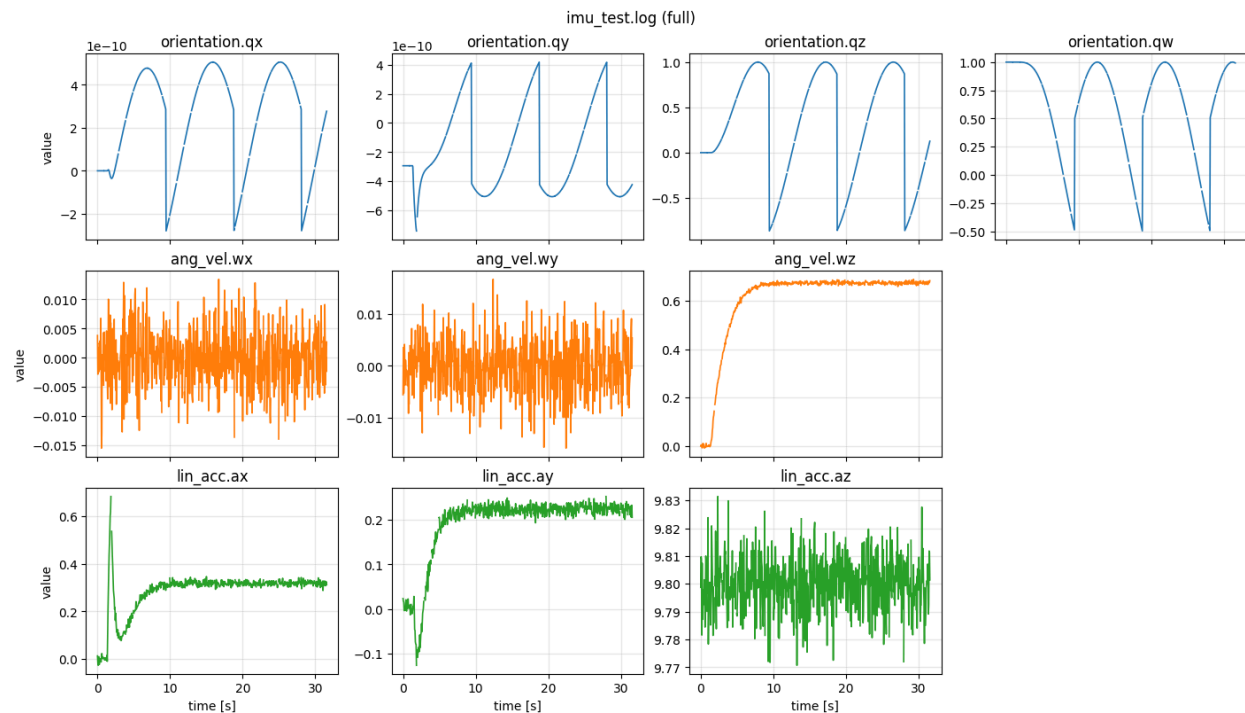


Figure 11: A 3x4 plot showing 30 seconds of IMU data. Plots should include orientation (roll, pitch, yaw), angular velocity (x, y, z), and linear acceleration (x, y, z).

i. Ability to define and incorporate “noise” to sensor outputs

Probabilistic robotics relies on modeling uncertainty, and Gazebo Sim allows for the injection of Gaussian noise directly at the sensor level. This is configured within the <sensor> block of the robot's SDF by adding a <noise> tag. We have configured this for our sensors, including the IMU and LIDAR. This is crucial for testing the robustness of filters against imperfect, real-world-style sensor data. Also in section for lidar shows this added sensor noise in action

Figure 12-13

For example, the IMU sensor's SDF block includes:

```
<sensor name="imu_sensor" type="imu">
  <always_on>1</always_on>
  <update_rate>100</update_rate>
  <visualize>true</visualize>
  <imu>
    <angular_velocity>
      <x>
```

```

<noise type="gaussian">
  <mean>0.0</mean>
  <stddev>0.002</stddev>
</noise>
</x>
<!-- ... noise added for y and z axes ... -->
</angular_velocity>
<linear_acceleration>
  <x>
    <noise type="gaussian">
      <mean>0.0</mean>
      <stddev>0.017</stddev>
    </noise>
  </x>
  <!-- ... noise added for y and z axes ... -->
</linear_acceleration>
</imu>
</sensor>

```

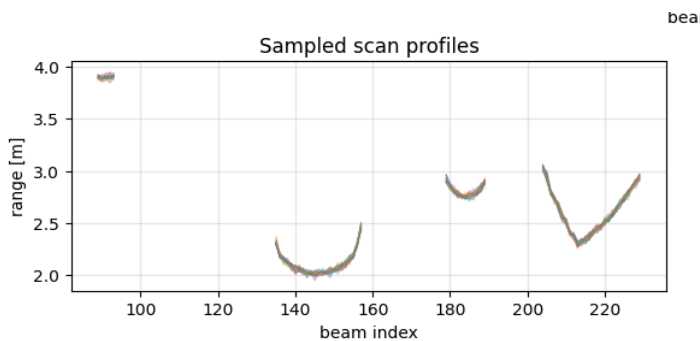
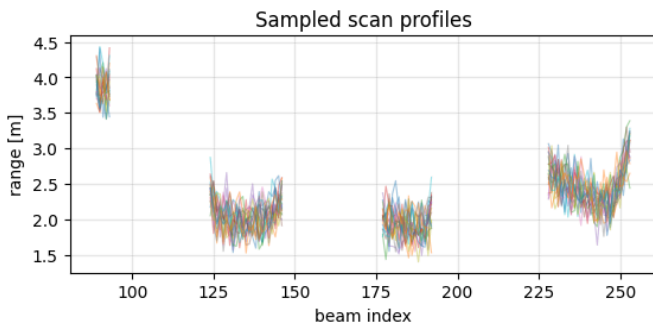


Figure 12 and 13 Showing the LIDAR readings from Figure 5 with high and low noise values. For low value it is 0.02 and high value is 0.2 set in the model's sdf.

Q2: Robot Following Task

Objective

To implement a follower robot (vehicle_green) that uses its onboard RGBD camera to track and maintain a set distance from a leader robot (vehicle_blue) in an obstacle-free environment. This task demonstrates the use of sensor data in a closed-loop feedback control system.

Algorithm / Control Logic

The solution is implemented as two separate ROS 2 nodes, demonstrating a modular perception and control pipeline. The camera_robot_checker.launch.py file launches both nodes.

1. Perception (camera_tracker_node)

This C++ node is responsible for identifying the leader robot and determining its position relative to the follower.

- **Sensing:** It subscribes to the follower's synchronized RGB (/camera/image) and depth (/camera/depth_image) camera streams.
- **Detection:** An OpenCV-based computer vision pipeline is used. The RGB image is converted to the HSV color space, and a binary mask is created by thresholding for a specific blue color range (HSV lower: [90, 80, 40], upper: [130, 255, 255]), corresponding to the vehicle_blue leader.
- **Localization:** The largest contour in the mask is found. If it exceeds a min_area threshold, its geometric centroid (cx, cy) is calculated.
- **Depth Estimation:** To find the distance, a small window (depth_win_x depth_win_y) around the centroid is sampled in the aligned depth image. The median depth value from this window is taken to reject outliers. This depth value is then smoothed using an Exponential Moving Average (EMA) (ema_alpha_ = 0.35) to reduce jitter from sensor noise.
- **Output:** The node publishes the leader's 2D pixel coordinate and 3D distance as a geometry_msgs/msg/Point message on the /target_pixel_coords topic, where Point.x = cx, Point.y = cy, and Point.z = smoothed_depth.

2. Control (green_follower_node)

This C++ node subscribes to the target data and implements the dual P-controller logic to command the follower robot's (vehicle_green) velocity.

- **Rotational Control (Alignment):** The controller seeks to center the leader in its field of view. The angular error is calculated as the normalized horizontal pixel offset:

$e_{ang} = (cx - image_center_x)/image_center_x$. The angular.z velocity is then set using a proportional controller: $v_{ang} = -k_{ang} \cdot e_{ang}$, where $k_{ang} = 1.2$. The negative sign ensures the robot turns *toward* the target.

- **Translational Control (Distance):** The controller seeks to maintain a fixed distance. The linear error is $e_{lin} = (measured_depth - goal_dist)$, where $goal_dist$ is set to 1.5m. The linear.x velocity is set by a second proportional controller: $v_{lin} = k_{lin} \cdot e_{lin}$, where $k_{lin} = 0.8$. To prevent collisions, the velocity is clamped at a max_lin of 0.8 m/s and is not allowed to be negative (i.e., the robot will not back up if it is too close, it will simply stop).
- **Search Behavior:** If a valid target message has not been received for $lost_timeout$ (0.5 seconds), the robot assumes the leader is lost. It stops its linear motion and rotates at a constant $search_omega$ (0.6 rad/s) to scan the environment until the leader is reacquired.

Performance Criteria

The goal of this task is to demonstrate a functional feedback loop. Performance is evaluated by analyzing and comparing:

1. **Commanded vs. Executed Velocities:** How well the robot's odometry-based velocities ($odom\ vx$, $odom\ wz$) follow the controller's commands ($cmd\ vx$, $cmd\ wz$).
2. **Controller Behavior:** Observing the P-controller's response (e.g., overshoot, settling time) to errors in alignment and distance.
3. **Odometric Drift:** The primary goal of probabilistic robotics is to correct for odometric drift. We will visualize this drift by comparing the robot's internally estimated Odometry path against its Ground Truth (GT) path from Gazebo.

Experimental Results & Data

To validate the controller and demonstrate the core concepts of odometric drift, two scenarios were executed. In both, the leader vehicle `blue` was driven in a simple path, and the follower vehicle `green` (running the controller) was started simultaneously. The resulting ground truth pose, odometry, and velocity commands for the follower (vehicle `green`) were logged from ROS 2 bags.

Scenario 1: Ideal Physics (No Motion Noise)

This scenario tests the controller and odometry system in a "perfect" world. The slip parameters in the SDF were set to zero, so the only source of error is the odometry plugin's own simulated encoder noise.

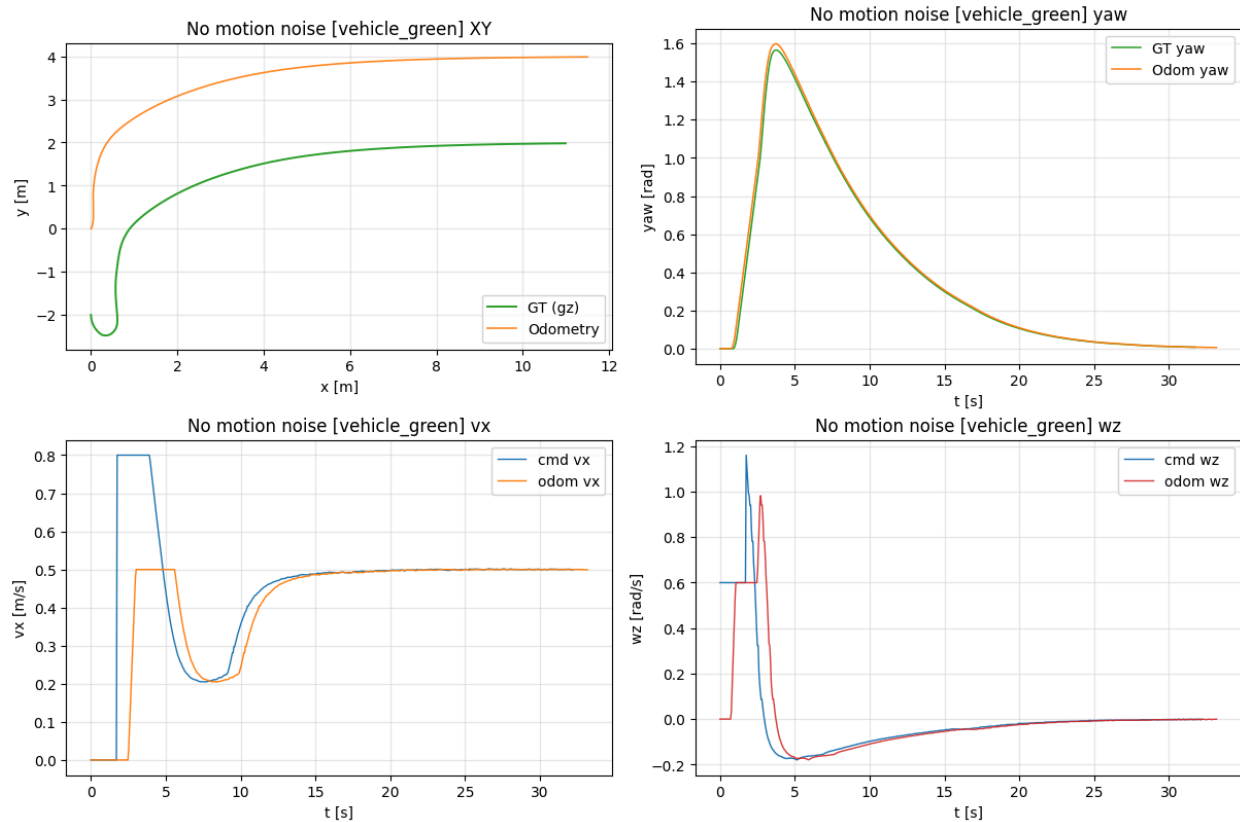


Figure 14: Follower robot data in an ideal simulation (no physical slip).

Analysis of Plots (Figure 14):

- XY Plot (Top-Left):** This plot perfectly illustrates the fundamental problem of localization. The green line, **GT (gz)**, is the robot's true physical path. The orange line, **Odometry**, is the path the robot *thinks* it has taken based on its internal wheel encoders. Even in this ideal world, the odometry path quickly drifts, accumulating significant position error by the end of the run.
- Yaw Plot (Top-Right):** This plot confirms the odometric drift. The GT yaw (green) and Odom yaw (orange) diverge, showing the robot's internal heading estimate is becoming progressively more inaccurate.
- Controller Plots (Bottom):** These plots show the P-controller behavior.
 - wz (Bottom-Right):** The cmd wz (blue line) is the angular velocity command from our P-controller. At the start ($t=0$ to $t=3s$), the robot searches for the target. Once detected, the controller sends a large angular command (~ 1.2 rad/s) to align. We see a classic **P-controller overshoot**, where the command goes negative to correct the turn, before oscillating and settling to a small value to maintain tracking.
 - vx (Bottom-Left):** The cmd vx (blue line) is the linear velocity command. It commands max speed (0.8 m/s) to close the initial distance, then drops

(overshoots) as it gets too close, finally settling at the leader's steady-state speed.

Scenario 2: Realistic Physics (With Motion Noise)

This scenario enables the "real" motion noise (wheel slip) in the Gazebo physics engine, as configured in Q1(c).

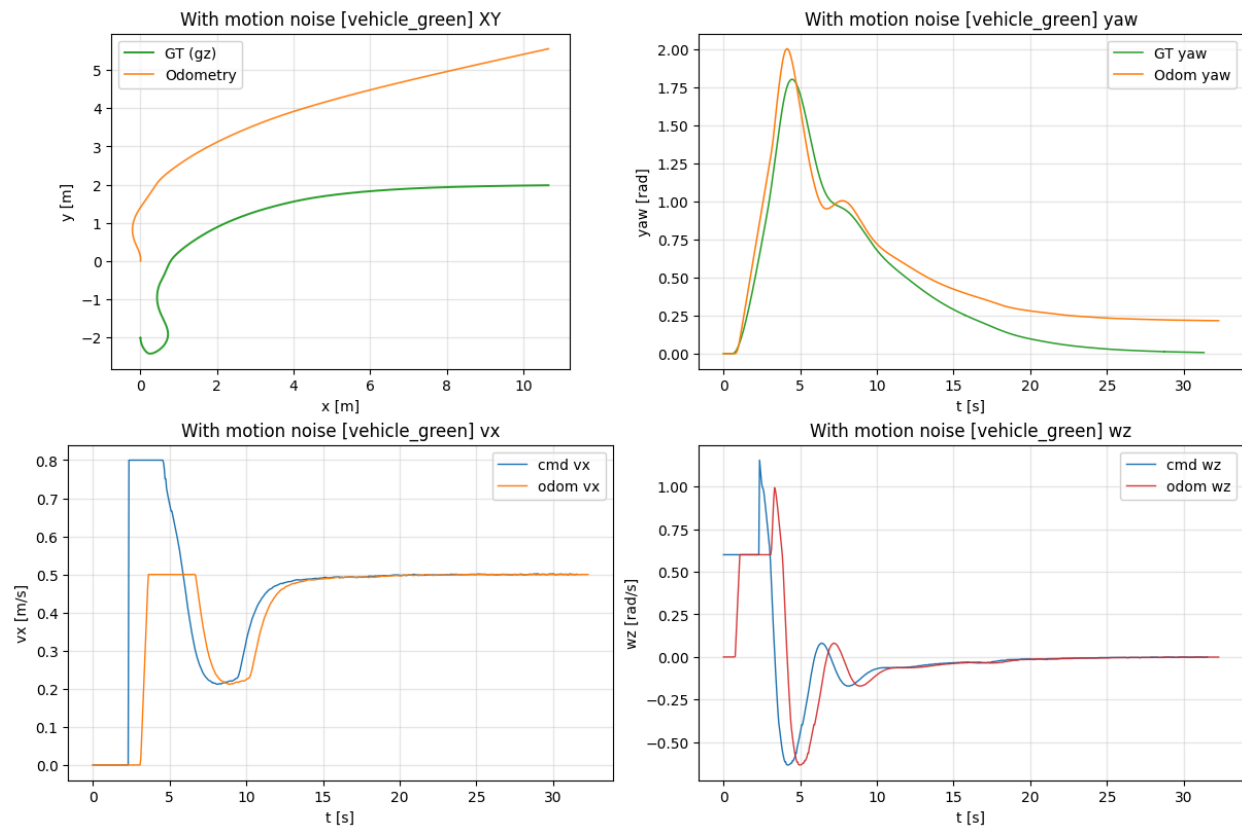


Figure 15: Follower robot data in a noisy simulation (physical slip enabled).

Analysis of Plots (Figure 15):

- XY Plot (Top-Left):** The situation is now worse, as expected. The **GT (gz)** path (green) is *itself* wobbly and deviates from the intended smooth curve due to the physical wheel slip. The **Odometry** path (orange) is even more erratic, as it now represents the *compounded error* of both the physical slip and the noisy odometry sensor.
- Controller Plots (Bottom):** The controller is visibly working harder.
 - wz (Bottom-Right):** The cmd wz (blue line) is much "noisier" and more active than in the ideal case. This is **expected and correct behavior**. It shows that the follower's camera is observing the leader's *real, wobbly* motion (caused by slip), and the P-controller is constantly issuing small, high-frequency corrections to maintain alignment.

- **Yaw (Top-Right):** The Odom yaw now diverges from the GT yaw even more dramatically, demonstrating how physical noise makes odometry-only navigation fail even faster.

Demonstration Video

A video (demo.mp4) is included with this submission to show the system in action. The video provides a split-screen view of:

1. **Gazebo Sim:** Showing the vehicle_green (follower) physically tracking the vehicle_blue (leader).
2. **RViz2:** Showing the follower's perspective, including the raw RGB feed from /camera/image, the depth image from /camera/depth_image, and the calculated /target_pixel_coords being used by the controller.

This combination of static plots and dynamic video fully demonstrates the capabilities of the simulator and the functionality of the feedback control system.